

International Islamic University Chittagong

Department of Computer Science and Engineering

Final Project Report

Course Code: CSE-3644

Course Title: Software Development 2 Lab

Project Title: ShikkaLink – Providing Learning
Excellence

Submitted By:

1. Maimuna Tabasum (C233460)
2. Tahasina Tasnim Afra (C233456)
3. Safiyat Sayma (C233475)

Section: 6BF

Supervised By:

Md. Sadman Hafiz

Lecturer

Department of Computer Science and Engineering
International Islamic University Chittagong (IIUC)

Date of Submission: 4 August 2026

Declaration

We hereby declare that the work presented in this report titled **ShikkaLink – Providing Learning Excellence**, submitted for the course **CSE-3644: Software Development 2 Lab**, is our original work carried out under the supervision of **Md. Sadman Hafiz**. This report has not been submitted, either in whole or in part, for the award of any other degree or diploma, and all sources of information used have been properly acknowledged.

Group Members:

- Maimuna Tabasum (C233460)
- Tahasina Tasnim Afra (C233456)
- Safiyat Sayma (C233475)

Acknowledgement

First and foremost, we express our gratitude to Almighty Allah for granting us the strength and guidance to complete this project.

We sincerely thank our supervisor, **Md. Sadman Hafiz**, Lecturer, Department of Computer Science and Engineering, International Islamic University Chittagong, for his valuable guidance, constructive feedback, and continuous support throughout this project.

We are also grateful to the students, tutors, and administrators from QuickStudy UK, Dhikka.com, 10 Minute School, and Caretutors who participated in our requirement elicitation process. Their insights and feedback played an important role in identifying user needs and shaping the development of ShikkaLink.

Finally, we thank the Department of Computer Science and Engineering, International Islamic University Chittagong, for providing the academic environment and support necessary to complete this work.

Abstract

ShikkaLink is a web-based online tutoring platform developed to connect students with verified tutors in a structured, secure, and reliable manner. The platform addresses common challenges faced by students and tutors when searching for tuition opportunities through informal methods such as social media groups and personal recommendations, where verification, transparency, and effective management are often lacking.

Requirements were gathered through interviews with students, tutors, and administrators from existing tutoring platforms, including Dhikka.com, QuickStudy UK, 10 Minute School, and Caretutors. The study identified major issues such as insufficient tutor verification, manual scheduling, limited tutor filtering options, lack of transparent booking management, and the absence of a centralized platform that effectively connects students, tutors, and administrators.

The system was developed using the Agile Scrum methodology across three development sprints. The frontend was built using Next.js, while the backend was developed with Node.js and Express.js. MySQL with Prisma ORM was used for database management. The implemented modules include secure authentication with role-based access (Student, Tutor, and Administrator), tutor verification, tutor search and filtering, a simplified booking system, and dedicated dashboards for each user role.

The system was evaluated using both black-box and white-box testing techniques. The developed platform successfully satisfies the primary project objectives by providing a functional prototype for tutor discovery and management. Although the frontend-backend integration of the booking module remains partially unresolved, it has been documented as an identified limitation and future improvement. This report presents the complete development process, including requirement engineering, system design, implementation, testing, deployment, and project evaluation.

Contents

Declaration	i
Acknowledgement	ii
Abstract	iii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Proposed Solution	2
1.4 Objectives	2
1.5 Scope and Limitations	2
1.6 Organisation of the Report	3
2 Requirement Engineering	4
2.1 The Site and Its Stakeholders	4
2.2 Requirement Elicitation	4
2.3 Study of the Existing System	5
2.4 Requirement Analysis	5
2.5 Functional Requirements	6
2.6 Non-Functional Requirements	6
2.7 Feasibility Study	7
2.8 Use Cases	7
2.9 Choice of Methodology	10
2.10 Scrum Roles	10
2.11 Scrum Events	10
2.12 Product Backlog	11
2.13 Sprint Plan	12
2.14 Sprint Backlogs	12

3	System Design	16
3.1	System Architecture	16
3.2	Database Design	16
3.3	Process Models	17
3.4	UI/UX Design	19
3.5	Technology Stack	20
4	Sprint Execution and Implementation	21
4.1	Sprint 1 – Requirement Engineering	21
4.1.1	Sprint Goal	21
4.1.2	Implementation	21
4.1.3	Sprint Outcome	21
4.2	Sprint 2 – Homepage Development	22
4.2.1	Sprint Goal	22
4.2.2	Implementation	22
4.2.3	Sprint Outcome	22
4.3	Sprint 3 – Authentication Module	22
4.3.1	Sprint Goal	22
4.3.2	Implementation	22
4.3.3	Sprint Outcome	22
4.4	Sprint 4 – Dashboard Module	23
4.4.1	Sprint Goal	23
4.4.2	Implementation	23
4.5	Sprint 5 – Tutor Verification Module	23
4.5.1	Sprint Goal	23
4.5.2	Implementation	23
4.6	Sprint 6 – Tutor Search Module	23
4.6.1	Sprint Goal	23
4.6.2	Implementation	23
4.7	Sprint 7 – Tutor Profile Module	24
4.7.1	Sprint Goal	24
4.7.2	Implementation	24
4.8	Sprint 8 – Booking Module and Final Integration	24
4.8.1	Sprint Goal	24
4.8.2	Booking Module	24
4.8.3	System Testing	24
4.8.4	Overall Sprint Outcome	24
4.9	Implementation Screenshots	25
4.10	GitHub Implementation	28

4.11	Sprint Review and Retrospective	28
5	Testing	29
5.1	Testing Strategy	29
5.2	Unit Testing	29
5.3	Black Box Testing	29
5.4	White Box Testing	30
5.5	Test Case Generation	30
5.6	Component Testing	31
5.7	End-to-End (E2E) Testing	31
5.8	Integration and System Testing	31
5.9	User Acceptance Testing	32
5.10	Bug Log	32
6	Deployment	33
6.1	Deployment Environment	33
6.2	Deployment Procedure	33
6.3	Live Links	34
6.4	User Manual	34
7	Conclusion	36
7.1	Achievement of Objectives	36
7.2	Challenges Faced	37
7.3	Future Work	37
7.4	Individual Contribution	38
	References	39
	Appendices	40

List of Figures

2.1	Entity Relationship Diagram of ShikkaLink	8
3.1	Entity Relationship Diagram of ShikkaLink	17
3.2	Data Flow Diagram (Level 0)	18
3.3	Data Flow Diagram (Level 1)	18
3.4	CRC Cards of ShikkaLink	19
4.1	Homepage of the ShikkaLink Tutor Hiring Platform	25
4.2	Find Tutors Page	26
4.3	Tutor Profile Page	26
4.4	Student Dashboard	27
4.5	Tutor Dashboard	27
4.6	Administrator Dashboard	28

List of Tables

2.1	Functional Requirements	6
2.2	Scrum Roles and Responsibilities	10
2.3	Product Backlog	11
2.4	Sprint Plan Overview	12
3.1	Technology Stack	20
5.1	Bug Log	32
7.1	Individual Contribution	38

1 Introduction

1.1 Background

Private tutoring has become an important part of the education system in Bangladesh, with many students seeking additional academic support outside the classroom. However, students and guardians still rely heavily on informal channels such as Facebook groups, personal recommendations, and tutoring agencies to find tutors. These methods often lack proper verification, transparency, and efficient management, making it difficult for users to find trustworthy tutors and manage tutoring sessions effectively.

1.2 Problem Statement

To understand the existing challenges in online tutoring platforms, interviews were conducted with students, tutors, and administrators associated with Dhikka.com, QuickStudy UK, 10 Minute School, and Caretutors. The findings revealed several common issues in the current tutor hiring process. Students reported difficulties in identifying qualified and verified tutors, comparing tutors based on qualifications, and booking sessions conveniently. Tutors highlighted the need for secure identity verification, better scheduling systems, fixed pricing, and location-based matching. Administrators emphasized the importance of structured tutor verification through interviews and assessments, proper profile evaluation, and centralized management of tutor applications. These observations indicated the need for a unified platform that simplifies tutor discovery, verification, booking, and management.

The primary problems identified are:

- Difficulty in finding verified and trustworthy tutors.
- Lack of a centralized platform for tutor search and booking.
- Manual and inefficient tutor verification procedures.
- Limited filtering options based on subject, location, curriculum, and teaching preferences.

- Absence of role-based management for students, tutors, and administrators.
- No centralized system for tracking tutor applications and booked sessions.

1.3 Proposed Solution

ShikkaLink was developed as a web-based tutoring platform that connects students with verified tutors through a secure and structured system. Students can search and filter tutors based on their requirements, while tutors undergo a verification process before becoming available for bookings. Administrators manage tutor verification requests and platform activities through a dedicated dashboard. The platform also provides secure authentication, role-based access control, tutor profile management, and a simplified booking workflow.

1.4 Objectives

1. To develop a secure authentication system using JWT and bcrypt password hashing for Students, Tutors, and Administrators.
2. To implement a tutor verification process involving profile review, subject assessment, and administrator approval.
3. To provide tutor searching and filtering based on relevant criteria such as subject and tutor profile.
4. To implement a simplified one-click booking system for students.
5. To develop dedicated dashboards for students, tutors, and administrators with role-based functionality.
6. To enable administrators to approve or reject tutor verification requests and allow users to manage their bookings.

1.5 Scope and Limitations

The current version of ShikkaLink includes secure user authentication, tutor verification, tutor search and filtering, a simplified booking process, and role-based dashboards for students, tutors, and administrators. Features such as real-time chat, video conferencing, online payment integration, AI-based tutor recommendations, advanced availability scheduling, and automated timetable conflict detection are beyond the scope of this version. Additionally, the frontend-backend integration of the booking

module is partially incomplete due to an unresolved authentication and routing issue, which has been documented for future development.

1.6 Organisation of the Report

Chapter 2 presents the requirement engineering process, including stakeholder interviews and requirement analysis. Chapter 3 describes the Agile Scrum methodology followed during development. Chapter 4 discusses the system architecture, database design, and user interface design. Chapter 5 explains the implementation completed in each sprint. Chapter 6 presents the testing process and results. Chapter 7 discusses deployment, while Chapter 8 concludes the report with project evaluation, limitations, and future work.

2 Requirement Engineering

2.1 The Site and Its Stakeholders

The requirements for ShikkaLink were gathered through interviews with stakeholders associated with existing tutoring platforms rather than from a single organisation. The elicitation process included discussions with:

- A student user of **Dhikka.com**
- An online and offline tutor associated with **Dhikka.com**
- An administrator and tutor from **QuickStudy UK**
- A tutor and student familiar with **10 Minute School**
- A verified tutor from **Caretutors**

The interviews were conducted during the requirement engineering phase of the project through online meetings and direct communication. These stakeholders represented the three primary user groups of the proposed system: students, tutors, and administrators. Their experiences with existing tutoring platforms helped identify the practical problems faced during tutor recruitment, verification, scheduling, and session management.

2.2 Requirement Elicitation

Requirements were collected primarily through semi-structured interviews with students, tutors, and administrators. The interviews focused on understanding the shortcomings of existing tutoring platforms and identifying features that users considered essential for a modern tutor hiring platform. Information gathered from different stakeholders was compared and consolidated to produce the final requirement specification.

The major interview questions included:

1. What benefits do you expect from an online tutoring platform?

2. Which features would encourage you to use such a platform regularly?
3. What were the strengths and weaknesses of platforms you previously used?
4. Would tutor searching and filtering improve your experience?
5. How should scheduling and booking be managed?
6. What type of tutor verification process would you prefer?
7. Do you have any additional suggestions for improving the platform?

Detailed interview notes and questionnaires are provided in the Appendix.

2.3 Study of the Existing System

The existing tutor recruitment process largely depends on Facebook groups, tutoring agencies, personal recommendations, and existing tutoring platforms. Students search for tutors manually, contact them through phone calls or messaging applications, negotiate fees, and arrange schedules individually. Verification of tutors is inconsistent across platforms, and there is no standardized method for managing tutor applications, bookings, or communication.

Common problems identified include:

- Difficulty verifying tutor qualifications and identity.
- Manual scheduling through phone calls or messaging applications.
- Limited filtering options when searching for tutors.
- Lack of centralized booking management.
- Poor transparency regarding tutor profiles and pricing.

2.4 Requirement Analysis

After collecting stakeholder feedback, similar requirements were grouped together into common functional areas including authentication, tutor verification, tutor search, booking, scheduling, and administration. Requirements mentioned by multiple stakeholders were assigned higher priority.

For example, almost every stakeholder emphasized the importance of tutor verification, secure authentication, and efficient tutor searching. Students prioritized usability, transparency, and filtering features, while tutors requested better scheduling and

profile management. Administrators focused on structured verification and centralized management of tutor applications.

Some requested features, including integrated video conferencing, online payment gateways, AI recommendations, and advanced timetable management, were considered beyond the scope of the current project due to development time constraints. These features have been identified as future enhancements.

2.5 Functional Requirements

Table 2.1: Functional Requirements

ID	Description	Priority	Stakeholder
FR-01	Users can register and log in as Student or Tutor, while administrators access a dedicated admin account.	High	Student, Tutor, Admin
FR-02	Tutors must complete verification before appearing in tutor search results.	High	Tutor, Admin
FR-03	Students can search and filter tutors based on subject, class, and other profile information.	High	Student
FR-04	Students can submit a booking request to a verified tutor.	High	Student
FR-05	Students and tutors can manage bookings through their respective dashboards.	Medium	Student, Tutor
FR-06	Administrators can review, approve, or reject tutor verification requests.	High	Admin
FR-07	Tutors can update their teaching profile and qualifications.	Medium	Tutor
FR-08	Users can securely log out of the system.	Medium	All Users

2.6 Non-Functional Requirements

- **Performance:** Tutor search results should be displayed within 2 seconds under normal operating conditions.
- **Security:** Passwords are encrypted using bcrypt hashing, while authenticated users access protected resources using JWT-based authentication.
- **Usability:** The platform should provide a simple, responsive interface that can be used easily on both desktop and mobile devices.

- **Availability:** The system should remain available throughout normal usage and recover quickly from server interruptions.
- **Compatibility:** The web application supports modern browsers including Google Chrome, Microsoft Edge, Firefox, and mobile browsers.

2.7 Feasibility Study

Technical Feasibility: The project was developed using technologies familiar to the development team, including Next.js, Node.js, Express.js, MySQL, Prisma ORM, and JWT authentication, making implementation technically feasible.

Operational Feasibility: The proposed platform satisfies the needs identified by students, tutors, and administrators by simplifying tutor discovery, verification, and booking through a centralized web application.

Economic Feasibility: Since all major development tools, frameworks, and databases are open-source, development costs remained low. Only standard computing resources and internet connectivity were required.

Schedule Feasibility: The project was completed using Agile Scrum over three development sprints. Core functionalities were successfully implemented within the allocated academic project timeline.

2.8 Use Cases

The major actors of the system are:

- Student
- Tutor
- Administrator

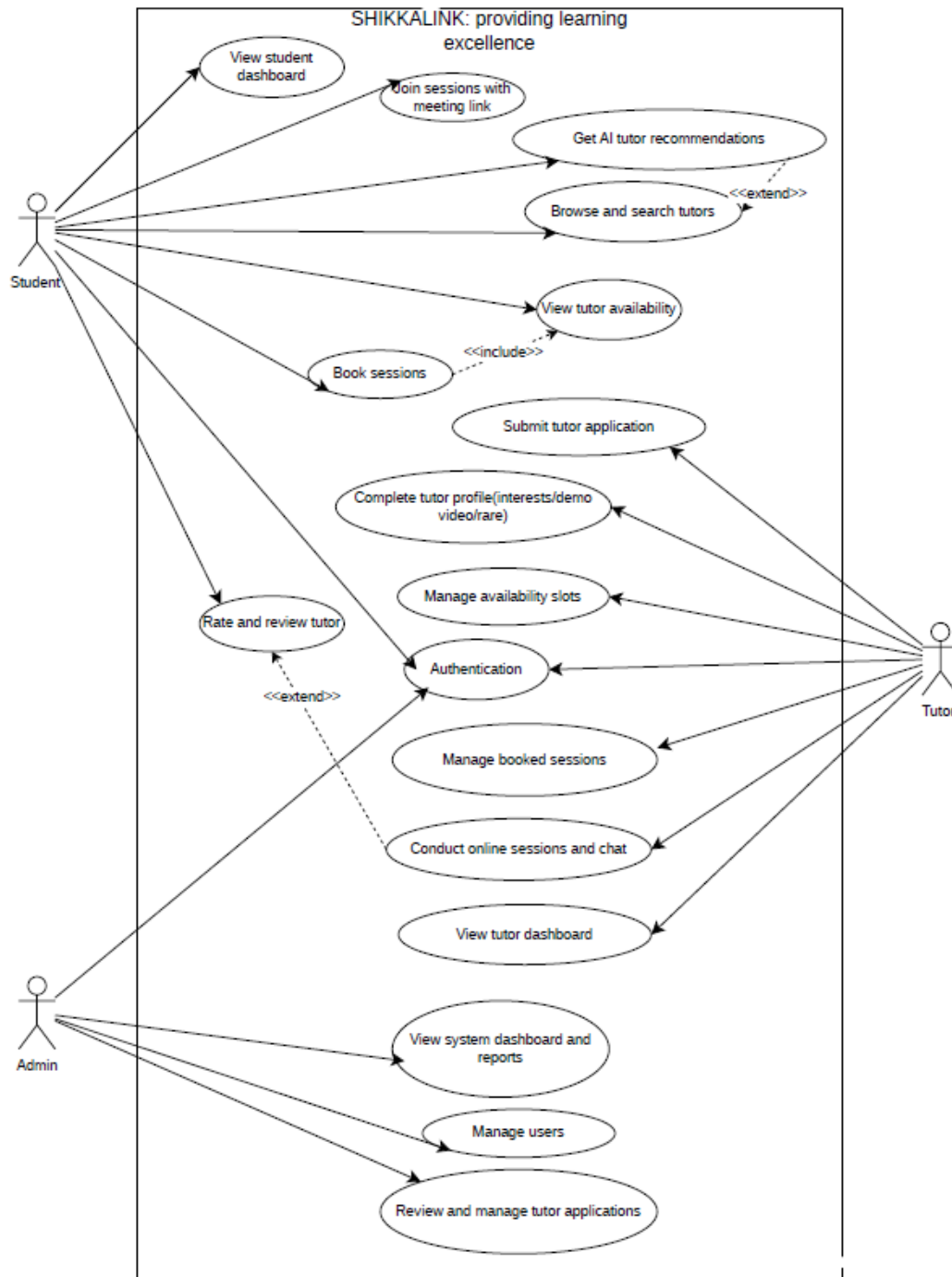


Figure 2.1: Entity Relationship Diagram of ShikkaLink

Use Case 1: Student Books a Tutor

Actor: Student

Preconditions:

- Student is logged into the system.
- Tutor has been verified by the administrator.

Main Flow:

1. Student searches for tutors.
2. Student filters available tutors.
3. Student views tutor profile.
4. Student submits a booking request.

Postcondition: The booking request is stored and displayed on the student dashboard.

Use Case 2: Tutor Submits Verification

Actor: Tutor

Preconditions:

- Tutor account has been created.

Main Flow:

1. Tutor completes profile information.
2. Tutor submits verification details.
3. Verification request is sent to the administrator.

Postcondition: Tutor remains pending until approved by the administrator.

Use Case 3: Administrator Verifies Tutor

Actor: Administrator

Preconditions:

- A tutor verification request exists.

Main Flow:

1. Administrator reviews tutor information.
2. Administrator approves or rejects the request.
3. System updates tutor verification status.

Postcondition: Approved tutors become visible in tutor search results, while rejected tutors remain unavailable.

2.9 Choice of Methodology

The Agile Scrum methodology was selected for the development of ShikkaLink because the project requirements evolved continuously throughout the development process based on stakeholder feedback collected from students, tutors, and administrators. Rather than implementing the entire system at once, the project was divided into smaller, manageable sprints, allowing each major feature to be developed, tested, and reviewed incrementally.

Compared to the Waterfall model, Scrum provided greater flexibility to accommodate requirement changes and integration issues discovered during development. Since the project was completed by a small team within a fixed academic timeline, Scrum enabled effective task distribution, continuous collaboration, and regular progress evaluation throughout the project.

2.10 Scrum Roles

The project followed the Scrum framework with responsibilities shared among team members while maintaining defined roles throughout development.

Table 2.2: Scrum Roles and Responsibilities

Role	Member	Responsibility
Product Owner	Team Members	Gathered stakeholder requirements, prioritized the product backlog, and finalized project features.
Scrum Master	Maimuna Tabasum	Coordinated sprint planning, monitored project progress, managed task allocation, and ensured smooth collaboration among team members.
Development Team	Maimuna Tabasum, Tahasina Tasnim Afra, Safiyat Sayma	Designed, implemented, integrated, tested, and documented the system collaboratively.

2.11 Scrum Events

The project was developed through multiple Scrum sprints. Before each sprint, the team conducted sprint planning sessions to determine the features to be implemented and assign responsibilities. Team members communicated regularly through online discussions and meetings to monitor progress, resolve implementation issues, and co-

ordinate frontend-backend integration.

At the end of each sprint, completed features were reviewed against the sprint goals, followed by discussions on improvements for the next sprint. This iterative process allowed the team to gradually enhance the system while addressing technical challenges encountered during development.

2.12 Product Backlog

Table 2.3: Product Backlog

User Story	Priority	Points
As a student, I want to register and log in so that I can securely access the platform.	High	5
As a tutor, I want to create an account and submit my profile for verification.	High	8
As a student, I want to search and filter verified tutors so that I can find a suitable tutor.	High	8
As a student, I want to view tutor profiles before making a booking.	High	5
As a student, I want to book a session with a verified tutor.	High	8
As a tutor, I want to manage my profile through my dashboard.	Medium	5
As an administrator, I want to approve or reject tutor verification requests.	High	8
As a user, I want to log out securely after using the system.	Medium	3

2.13 Sprint Plan

Table 2.4: Sprint Plan Overview

Sprint	Sprint Goal	Module Delivered
1	Gather requirements and analyze stakeholder needs.	Requirement Engineering
2	Design and implement the landing page.	Homepage Module
3	Develop user authentication and role-based login.	Authentication Module
4	Develop dashboards for students, tutors, and administrators.	Dashboard Module
5	Implement tutor verification workflow.	Tutor Verification Module
6	Develop tutor search and filtering functionality.	Tutor Search Module
7	Implement tutor profile viewing functionality.	Tutor Profile Module
8	Implement booking functionality and perform final integration and testing.	Booking Module (Partially Functional)

2.14 Sprint Backlogs

Sprint 1 – Requirement Engineering

Goal: Gather stakeholder requirements and define the project scope.

- Stakeholder interviews
- Requirement analysis
- Product backlog preparation

Team Contribution

- Maimuna Tabasum – Team coordination, requirement planning, and project management.

- Tahasina Tasnim Afra – Requirement analysis and backend planning.
- Safiyat Sayma – UI research and documentation.

Sprint 2 – Homepage

Goal: Develop the homepage and overall user interface.

- Homepage design
- Navigation
- Responsive UI

Team Contribution

- Maimuna Tabasum – Team coordination, frontend-backend integration planning, and testing.
- Tahasina Tasnim Afra – Backend setup and API preparation.
- Safiyat Sayma – Homepage frontend development and UI implementation.

Sprint 3 – Authentication

Goal: Implement secure user authentication.

- Registration
- Login
- JWT authentication
- Role-based authorization

Team Contribution

- Maimuna Tabasum – Team coordination, integration, and testing.
- Tahasina Tasnim Afra – Authentication backend, APIs, and database integration.
- Safiyat Sayma – Authentication frontend and UI.

Sprint 4 – Dashboards

Goal: Develop dashboards for all user roles.

Team Contribution

- Maimuna Tabasum – Team coordination, frontend-backend integration, and testing.
- Tahasina Tasnim Afra – Dashboard backend APIs and integration.
- Safiyat Sayma – Dashboard frontend implementation and UI.

Sprint 5 – Tutor Verification

Goal: Implement tutor verification workflow.

Team Contribution

- Maimuna Tabasum – Team coordination, integration, and testing.
- Tahasina Tasnim Afra – Verification backend APIs and database integration.
- Safiyat Sayma – Verification frontend interface and UI.

Sprint 6 – Tutor Search

Goal: Develop tutor searching and filtering.

Team Contribution

- Maimuna Tabasum – Team coordination, integration, and testing.
- Tahasina Tasnim Afra – Search backend APIs and filtering logic.
- Safiyat Sayma – Tutor search frontend and UI.

Sprint 7 – Tutor Profile

Goal: Implement tutor profile viewing.

Team Contribution

- Maimuna Tabasum – Team coordination, integration, and testing.
- Tahasina Tasnim Afra – Backend APIs for tutor profiles.
- Safiyat Sayma – Tutor profile frontend and UI implementation.

Sprint 8 – Booking and Final Integration

Goal: Implement booking functionality and complete system integration.

Team Contribution

- Maimuna Tabasum – Team coordination, frontend-backend integration, module merging, and final system testing.
- Tahasina Tasnim Afra – Booking backend development and API integration.
- Safiyat Sayma – Booking frontend implementation and UI refinement.

The booking module was partially implemented. While the frontend interface and backend APIs were developed, complete frontend-backend integration could not be achieved due to unresolved authentication and routing issues. This limitation has been documented and recommended as future work.

3 System Design

3.1 System Architecture

ShikkaLink follows a three-tier client-server architecture consisting of the presentation layer, application layer, and database layer.

- **Frontend:** The user interface was developed using Next.js (React). It provides separate interfaces for students, tutors, and administrators and communicates with the backend through RESTful API requests.
- **Backend:** The application logic was implemented using Node.js with Express.js. The backend handles user authentication, tutor verification, tutor searching, booking requests, dashboard operations, and communication with the database.
- **Database:** MySQL was used as the relational database, while Prisma ORM was used to simplify database operations, manage relationships, and generate type-safe queries.
- **Authentication:** User authentication is implemented using JSON Web Tokens (JWT). Passwords are securely stored using bcrypt hashing, ensuring that sensitive user information is protected.

The frontend sends HTTP requests to the backend APIs, where business logic is executed. The backend then communicates with the MySQL database through Prisma ORM to retrieve or update data before returning responses to the client. This architecture separates the presentation, business logic, and data layers, making the system easier to maintain, extend, and test.

3.2 Database Design

The database was designed to support the core functionalities of ShikkaLink, including user registration, tutor verification, tutor searching, profile management, and

booking. The primary entities are **User**, **Tutor Profile**, **Tutor Verification**, and **Booking**. Relationships between these entities ensure data consistency while minimizing redundancy through normalization.

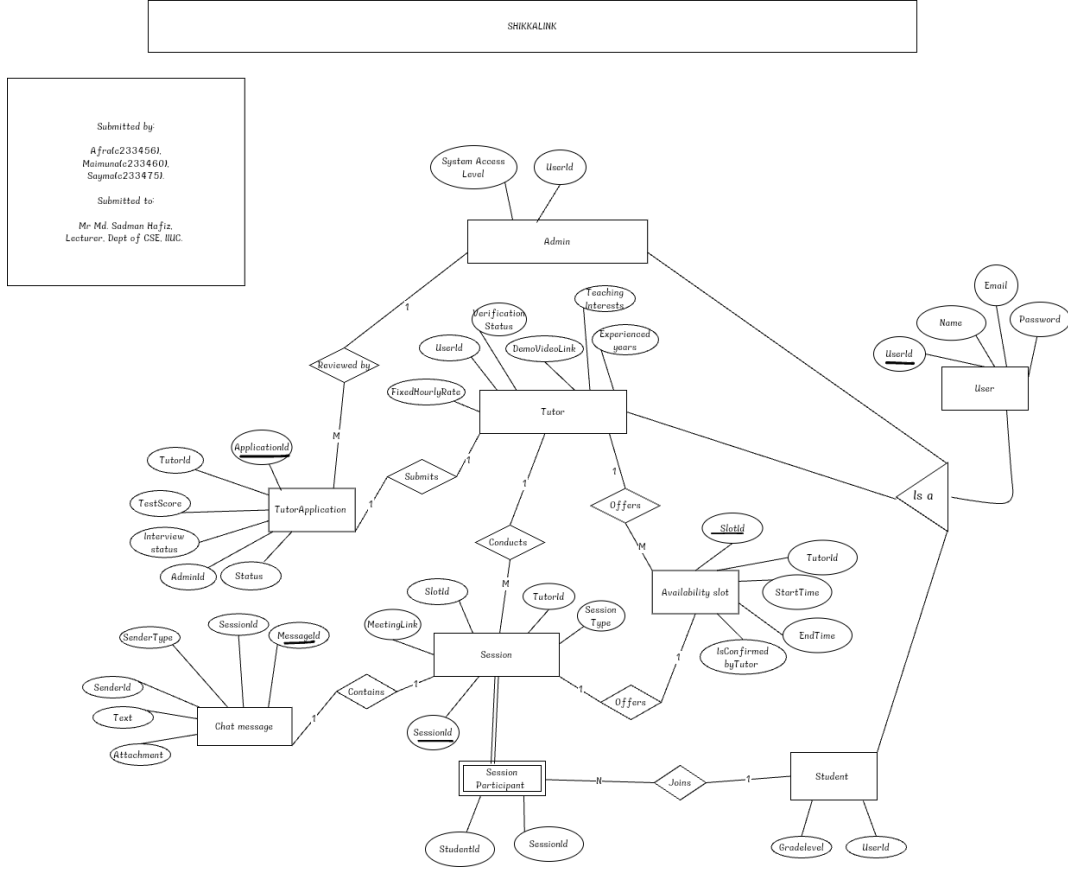


Figure 3.1: Entity Relationship Diagram of ShikkaLink

The database schema was implemented using Prisma ORM, which automatically generated the corresponding MySQL tables and maintained relationships using primary and foreign keys. The database design supports secure authentication, tutor profile management, verification workflows, and booking operations while maintaining referential integrity.

3.3 Process Models

The system processes were modeled using Data Flow Diagrams (DFDs) to illustrate the movement of data between users, the application, and the database.

Figure 3.2 presents the Level 0 DFD (Context Diagram), showing the overall interaction between students, tutors, administrators, and the ShikkaLink system.

Figure 3.3 presents the Level 1 DFD, which further decomposes the internal processes such as authentication, tutor verification, tutor search, booking, and dashboard management.

DFD-0 For ShikkaLink

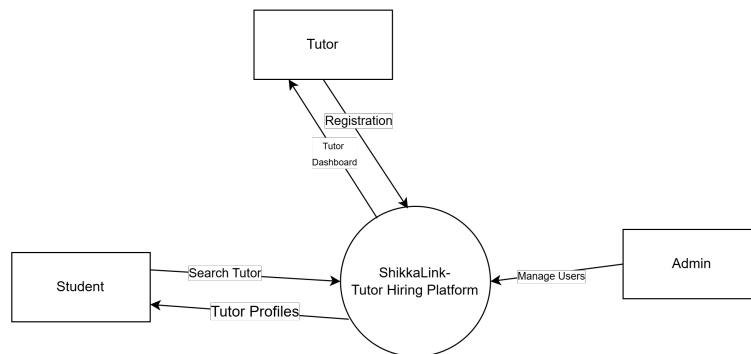


Figure 3.2: Data Flow Diagram (Level 0)

DFD-1

ShikkaLink- Tutor Hiring Platform

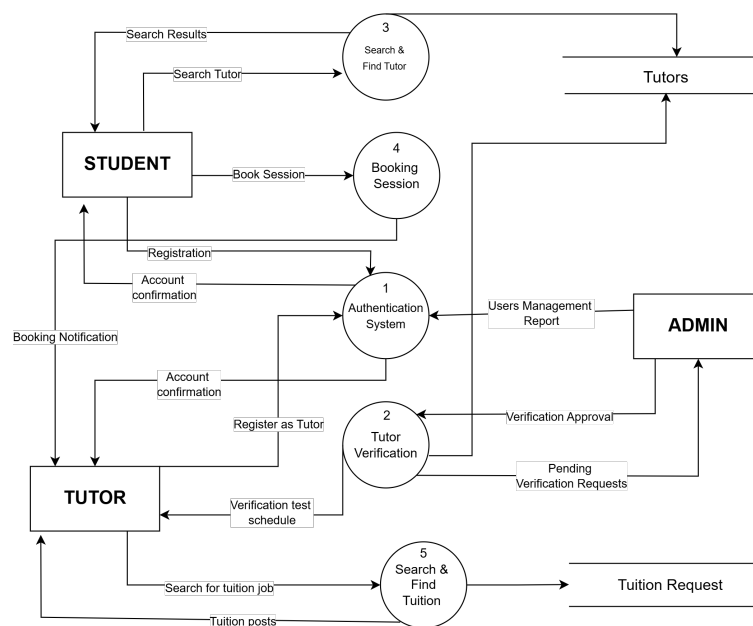


Figure 3.3: Data Flow Diagram (Level 1)

CRC cards were also prepared to define the responsibilities and collaborations of the major classes used in the system, including **User**, **Student**, **Tutor**, **Administrator**, **Tutor Verification**, and **Booking**. These helped organize the object-oriented design before implementation.

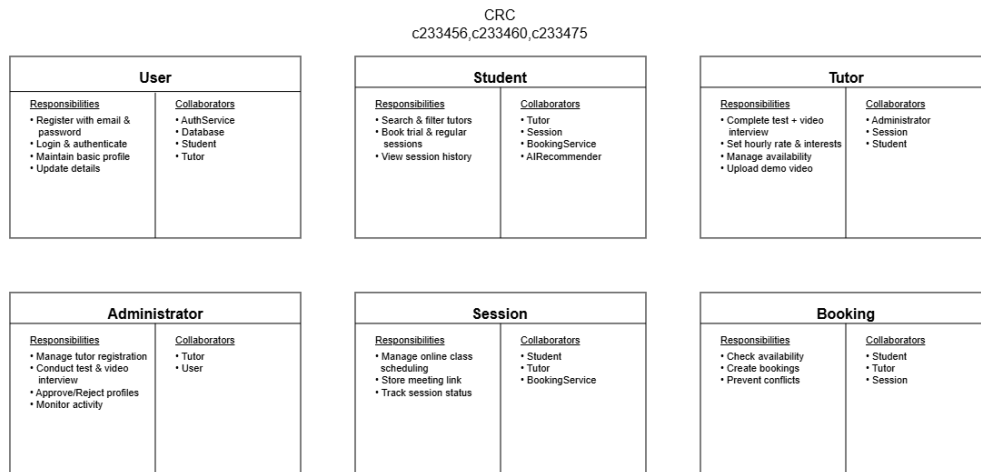


Figure 3.4: CRC Cards of ShikkaLink

3.4 UI/UX Design

The user interface was designed in Figma before implementation. Low-fidelity wireframes were prepared during the initial design stage to determine page layouts and navigation flow. These were later refined into high-fidelity prototypes that closely resemble the implemented application.

The design emphasizes:

- Simple and intuitive navigation.
- Responsive layouts for desktop and mobile users.
- Consistent color palette and typography.
- Clear call-to-action buttons.
- Easy tutor search and booking process.

The design decisions were influenced by stakeholder feedback collected during requirement engineering. Students requested a clean interface with easy tutor searching, while tutors emphasized simple profile management and verification workflows.

Figma Prototype:

<https://www.figma.com/design/CB0oTWV4fUsS0Ge105rG28/Shikka-link-we-ui?node-id=0-1&t=0AkjiqdA4hhfFJQt-1>

3.5 Technology Stack

Table 3.1: Technology Stack

Layer	Technology	Justification
Frontend	Next.js (React)	Component-based framework for developing responsive and interactive user interfaces.
Backend	Node.js + Express.js	Lightweight runtime and web framework for developing RESTful APIs.
Database	MySQL + Prisma ORM	Relational database with type-safe ORM for efficient database management.
Authentication	JWT + bcrypt	Secure token-based authentication with encrypted password storage.
Hosting	Local Development Environment	The system was developed, integrated, and tested on a local development server during the project.

4 Sprint Execution and Implementation

This chapter describes the implementation completed during each development sprint. The project followed an Agile Scrum methodology and was implemented incrementally, allowing each major module to be developed, tested, and integrated before moving to the next feature.

4.1 Sprint 1 – Requirement Engineering

4.1.1 Sprint Goal

The objective of this sprint was to gather stakeholder requirements, identify system problems, and define the project scope.

4.1.2 Implementation

The team interviewed students, tutors, and administrators from existing tutoring platforms including Dhikka.com, QuickStudy UK, 10 Minute School, and Caretutors. Based on the collected requirements, the product backlog and initial system specifications were prepared.

4.1.3 Sprint Outcome

- Requirement specification completed.
- Product backlog prepared.
- Initial project planning completed.

4.2 Sprint 2 – Homepage Development

4.2.1 Sprint Goal

Design and implement the homepage and overall navigation of the platform.

4.2.2 Implementation

A responsive landing page was developed using Next.js. The homepage introduces the platform and provides navigation to registration, login, tutor searching, and other major features.

4.2.3 Sprint Outcome

- Homepage completed.
- Responsive navigation implemented.
- Initial UI established.

4.3 Sprint 3 – Authentication Module

4.3.1 Sprint Goal

Develop secure authentication and role-based access.

4.3.2 Implementation

Users can register and log in as Students or Tutors, while administrators access the system through dedicated administrator accounts. Passwords are securely hashed using bcrypt before being stored in the database. JWT authentication is used to secure protected routes, and authenticated users are redirected to their respective dashboards based on their assigned roles.

4.3.3 Sprint Outcome

- User registration completed.
- Login implemented.
- JWT authentication completed.
- Role-based routing implemented.

4.4 Sprint 4 – Dashboard Module

4.4.1 Sprint Goal

Develop dashboards for Students, Tutors, and Administrators.

4.4.2 Implementation

Dedicated dashboards were developed for each user role.

- Student Dashboard displays booking information.
- Tutor Dashboard displays received booking requests.
- Admin Dashboard provides platform statistics and tutor verification management.

4.5 Sprint 5 – Tutor Verification Module

4.5.1 Sprint Goal

Implement tutor verification before tutors become publicly searchable.

4.5.2 Implementation

Tutors submit their information for verification. Administrators review submitted information and approve or reject tutor applications. Only verified tutors are displayed to students during tutor search.

4.6 Sprint 6 – Tutor Search Module

4.6.1 Sprint Goal

Develop tutor searching and filtering functionality.

4.6.2 Implementation

Students can search verified tutors using search functionality. Tutor cards display profile information, allowing students to browse suitable tutors before viewing complete profiles.

4.7 Sprint 7 – Tutor Profile Module

4.7.1 Sprint Goal

Implement detailed tutor profile pages.

4.7.2 Implementation

Students can access detailed tutor profiles containing tutor information, qualifications, experience, and teaching subjects before making booking decisions.

4.8 Sprint 8 – Booking Module and Final Integration

4.8.1 Sprint Goal

Complete booking functionality, integrate all modules, and perform final testing.

4.8.2 Booking Module

A simplified booking workflow was implemented. When a logged-in student selects a verified tutor and clicks the **Book Session** button, the system is designed to create a booking record without requiring availability scheduling or time-slot selection.

Although both the frontend interface and backend APIs were implemented, complete integration could not be achieved. During testing, booking requests occasionally failed due to unresolved authentication token validation and frontend-backend routing issues. As a result, the booking module remains partially functional and has been documented as future work.

4.8.3 System Testing

The completed modules were tested using both black-box and white-box testing techniques. Authentication, tutor verification, tutor searching, dashboards, and profile viewing operated successfully. The booking module requires additional integration work before deployment.

4.8.4 Overall Sprint Outcome

The final prototype successfully implements:

- User authentication

- Tutor verification
- Tutor searching
- Tutor profile viewing
- Student, Tutor, and Admin dashboards

The booking functionality remains the only partially completed feature due to unresolved integration issues.

4.9 Implementation Screenshots

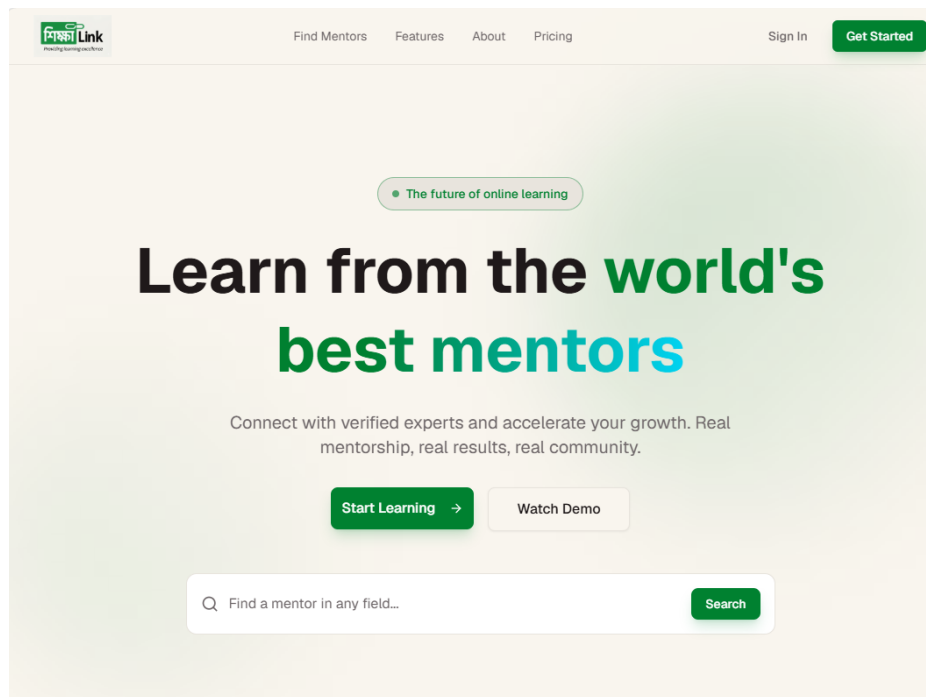


Figure 4.1: Homepage of the ShikkaLink Tutor Hiring Platform

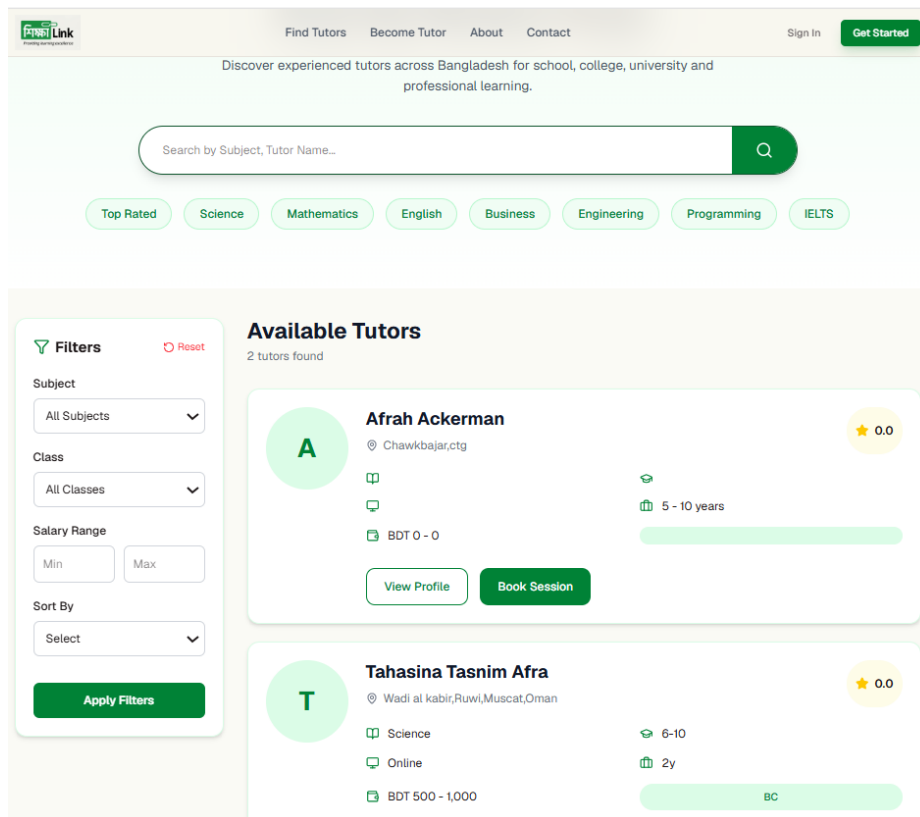


Figure 4.2: Find Tutors Page

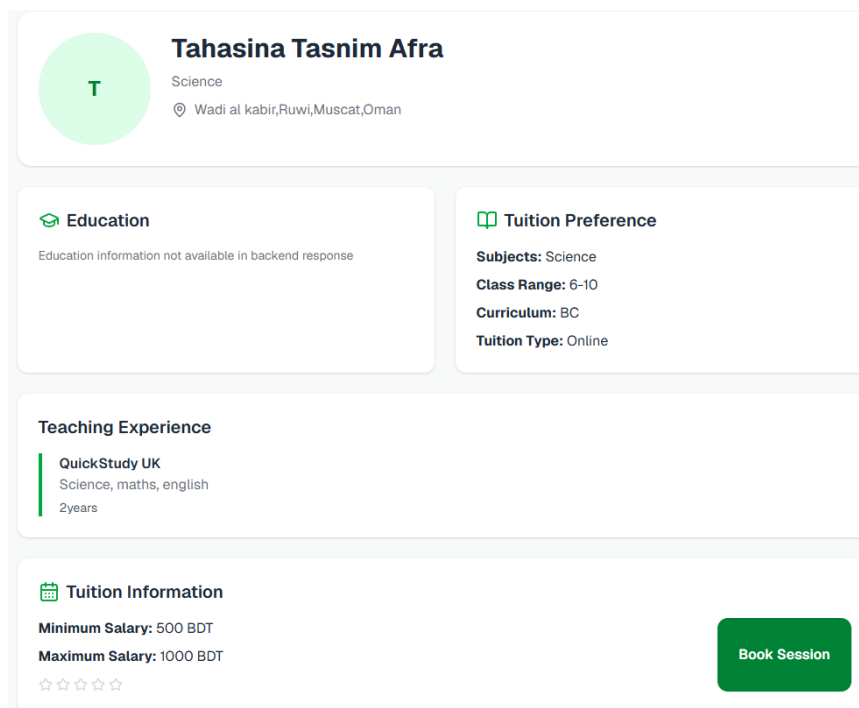


Figure 4.3: Tutor Profile Page

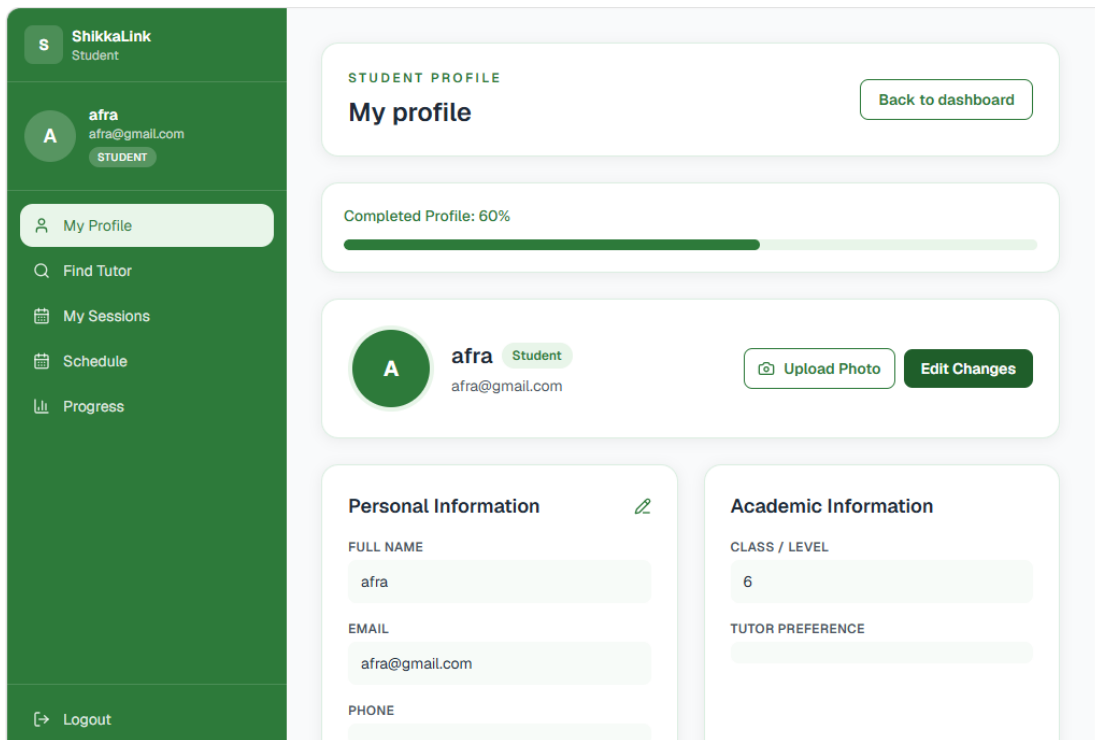


Figure 4.4: Student Dashboard

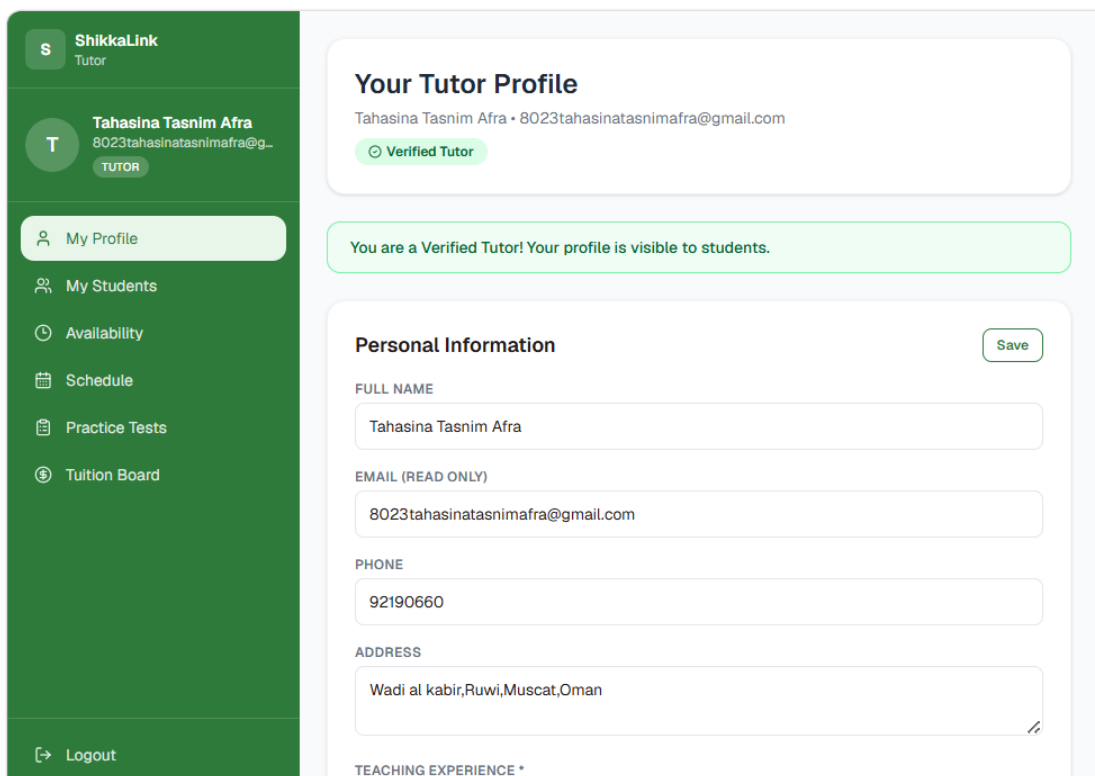


Figure 4.5: Tutor Dashboard

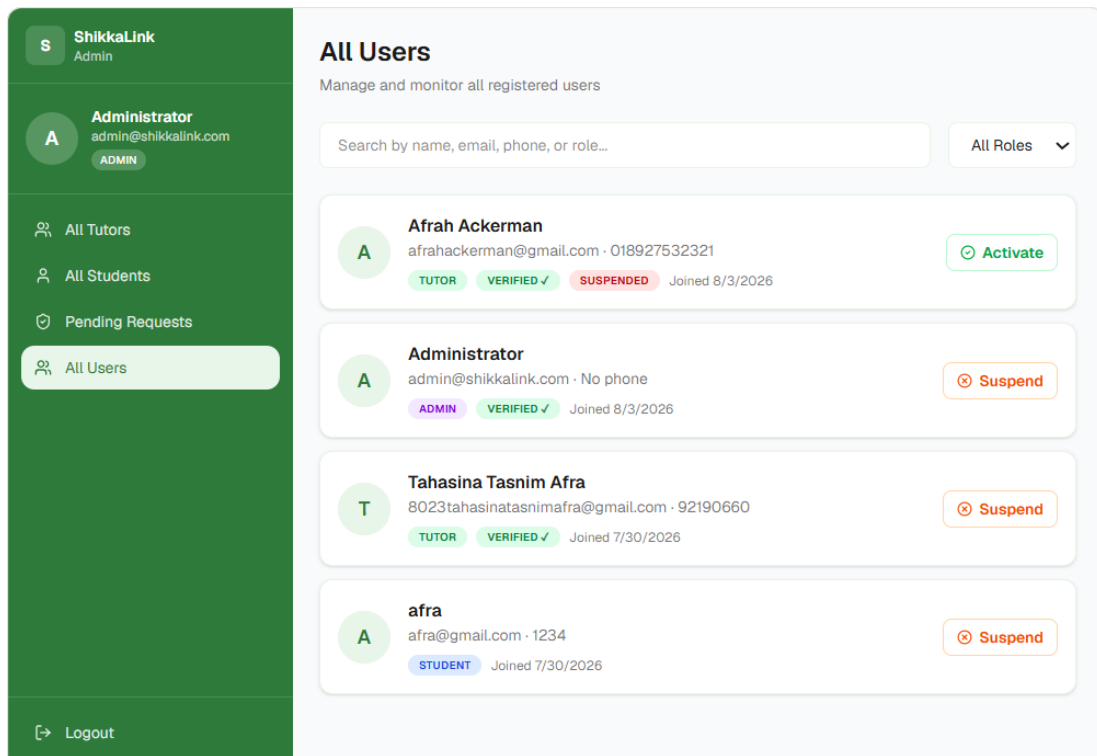


Figure 4.6: Administrator Dashboard

These interfaces demonstrate the core functionality of the platform and reflect the implementation completed during the development sprints.

4.10 GitHub Implementation

Version control was maintained throughout the project using Git and GitHub. Development was carried out through multiple branches corresponding to different modules and development tasks before being merged into the integration branch. GitHub commit history records the contributions of all team members and the incremental progress made during each sprint.

4.11 Sprint Review and Retrospective

At the end of each sprint, completed features were reviewed against the planned goals, and feedback was incorporated into subsequent sprints. Agile Scrum enabled the team to develop the system incrementally, collaborate effectively, and adapt to implementation challenges. Most planned modules were completed successfully, while the booking module remains future work due to unresolved frontend-backend integration issues.

5 Testing

5.1 Testing Strategy

Testing was performed throughout the development process to verify that the implemented features satisfied the specified functional requirements. The primary focus was on validating user interactions, authentication, role-based access control, tutor searching, dashboards, and booking functionality. Manual testing was carried out using both valid and invalid inputs to evaluate the correctness of the system's responses and identify implementation issues.

5.2 Unit Testing

Unit Testing is a software testing technique in which individual components or functions of a software system are tested independently before integration. The objective is to verify that each unit performs its intended functionality correctly according to its specification.

Typically, unit testing checks individual methods, functions, or modules by providing predefined inputs and comparing the actual output with the expected output. This approach helps developers detect logical errors at an early stage and improves the maintainability and reliability of the software.

5.3 Black Box Testing

Black Box Testing is a software testing technique in which the internal implementation of the software is not considered. Testing is performed solely based on system inputs and expected outputs from the user's perspective.

For ShikkaLink, black box testing was used to validate the major functional modules including user registration, login, tutor searching, tutor verification, dashboard access, and booking functionality. Both valid and invalid inputs were supplied to verify whether the system produced the expected results and displayed appropriate validation messages.

Examples of tested scenarios include:

- Registering users with valid information.
- Attempting registration using an existing email address.
- Entering invalid or incomplete input values.
- Logging in with correct and incorrect credentials.
- Verifying that users are redirected to the correct dashboard according to their assigned role.
- Searching tutors using valid and invalid search keywords.

Black box testing ensured that the implemented features behaved correctly from an end-user perspective.

5.4 White Box Testing

White Box Testing, also known as structural testing, evaluates the internal logic and implementation of software. The tester has knowledge of the source code and verifies the correctness of conditions, loops, data flow, and execution paths.

In ShikkaLink, white box testing was mainly applicable to backend modules such as user authentication, JWT token validation, role-based authorization, tutor verification logic, and booking APIs. Examining the application logic helped identify coding errors, authentication issues, and routing inconsistencies during development.

5.5 Test Case Generation

Test Case Generation involves designing test cases that verify whether the implemented system satisfies both functional and non-functional requirements. Each test case specifies the input, expected behavior, and actual outcome after execution.

For ShikkaLink, test cases were designed for both positive and negative scenarios. Valid inputs were used to verify successful execution, while invalid inputs were used to evaluate error handling and validation mechanisms.

Major test scenarios included:

- User registration with valid and invalid information.
- Duplicate email registration attempts.
- Login using correct and incorrect credentials.

- Dashboard access based on user roles.
- Tutor searching and filtering.
- Tutor verification workflow.
- Booking request submission.

These test cases helped verify that the implemented functionality behaved as expected under different operating conditions.

5.6 Component Testing

Component Testing focuses on verifying the functionality of individual modules before they are integrated into the complete system. Each component is tested independently to ensure that it performs its intended task correctly.

For ShikkaLink, component testing was carried out on major modules such as user authentication, tutor verification, tutor search, dashboards, and tutor profile management. Each component was tested using different input scenarios to verify that it produced the expected output before being integrated with other modules.

5.7 End-to-End (E2E) Testing

End-to-End (E2E) Testing verifies the complete workflow of the application from the user's perspective. It ensures that multiple modules interact correctly and that users can successfully perform common tasks from start to finish.

For ShikkaLink, end-to-end testing focused on user workflows such as registration, login, dashboard redirection, tutor searching, viewing tutor profiles, and tutor verification. These workflows were executed manually to confirm that the frontend, backend, and database communicated correctly. The booking workflow was also tested; however, complete end-to-end execution could not be achieved because of unresolved frontend-backend integration issues affecting the booking module.

5.8 Integration and System Testing

Integration testing was performed after individual modules had been implemented to verify that they communicated correctly with one another. Authentication was integrated with role-based dashboards to ensure that users were redirected to the appropriate interface after login. Tutor verification was integrated with the tutor search module so that only verified tutors appeared in search results.

System testing was conducted on the complete application to verify that all implemented modules worked together as an integrated system. Authentication, dashboards, tutor searching, tutor verification, and profile viewing operated successfully. The booking module remained partially functional because of unresolved frontend-backend authentication and routing issues, which prevented some booking requests from completing successfully.

5.9 User Acceptance Testing

The implemented prototype was demonstrated to team members and compared with the requirements gathered during stakeholder interviews. The completed modules generally satisfied the expected functionality for authentication, tutor verification, tutor searching, dashboards, and tutor profile management.

Feedback primarily focused on improving usability, simplifying navigation, and completing the booking functionality. These suggestions have been identified as future improvements for subsequent versions of the system.

5.10 Bug Log

The defects identified during development are summarized in Table 5.1. Most issues were resolved during implementation, while the booking module remains an open issue because of frontend-backend integration problems.

Table 5.1: Bug Log

Description	Severity	Sprint	Status	Resolution
Role-based dashboard redirection issue after login	Medium	3	Resolved	Updated authentication and routing logic.
Tutor search displayed unverified tutors	Medium	6	Resolved	Added verification status filtering.
Dashboard data synchronization issue	Low	4	Resolved	Updated API response handling.
Booking requests fail with fetch/access-denied errors due to authentication token and route mismatch	High	8	Open	Planned for future development.

6 Deployment

6.1 Deployment Environment

The frontend of the ShikkaLink Tutor Hiring Platform was deployed using Vercel. The application was developed with Next.js and TypeScript and connected to a GitHub repository for version control.

Throughout development, the backend API and MySQL database were executed locally using Express.js and Prisma ORM. Due to the limited project timeline, full deployment of the backend and database was not completed before the submission of this project. Therefore, only the frontend application was deployed online.

The technologies used during deployment are summarized below:

- Frontend Hosting: Vercel
- Frontend Framework: Next.js with TypeScript
- Backend Framework: Express.js (Local Development)
- Database: MySQL with Prisma ORM (Local Development)
- Version Control: Git and GitHub
- Development Environment: Windows 11

The frontend is connected to the GitHub repository, allowing automatic deployment whenever changes are pushed to the selected production branch.

6.2 Deployment Procedure

The frontend deployment followed the steps below:

1. The latest frontend source code was committed and pushed to the GitHub repository.
2. The application was tested locally to ensure that all dependencies were installed correctly and that the project built successfully.

3. A Vercel account was connected to the GitHub repository.
4. The frontend repository was imported into Vercel.
5. Vercel automatically detected the project as a Next.js application and applied the appropriate build configuration.
6. The required environment variables were configured in the Vercel project settings.
7. The frontend application was deployed and verified by testing navigation, page rendering, and responsive layouts.
8. Future frontend updates can be deployed automatically by pushing new commits to the connected GitHub branch.

6.3 Live Links

The following project resources are available:

- Frontend Deployment (Vercel): [ShikkaLink Live Demo](#)
- GitHub Repository: [ShikkaLink GitHub Repository](#)
- Figma Design: [ShikkaLink Figma Prototype](#)

6.4 User Manual

ShikkaLink provides separate interfaces for Students, Tutors, and Administrators. Each user accesses the system according to their assigned role after successful authentication.

Student

Students can:

- Register and log into the platform.
- Search and browse verified tutors.
- View tutor profiles.
- Send booking requests.
- View and manage their dashboard.

Tutor

Tutors can:

- Register for the platform.
- Complete their tutor profile.
- Submit their profile for verification.
- View booking requests through the tutor dashboard.
- Manage their profile information.

Administrator

Administrators can:

- Access the administrator dashboard.
- Review tutor verification requests.
- Approve or reject tutor applications.
- View platform statistics and manage tutor records.

The deployed frontend demonstrates the implemented user interface and navigation of the system. Full online functionality requires deployment of the backend services and database, which remains as future work.

7 Conclusion

7.1 Achievement of Objectives

The primary objective of ShikkaLink was to develop a web-based tutoring platform that provides a structured and secure environment for connecting students with verified tutors. Most of the objectives defined at the beginning of the project were successfully achieved.

- **Objective 1:** Develop a secure authentication system using JWT and bcrypt for Students, Tutors, and Administrators.
Status: Achieved. Secure registration, login, password hashing, and role-based authentication were successfully implemented.
- **Objective 2:** Implement a tutor verification process.
Status: Achieved. Tutors can submit their information for verification, and administrators can approve or reject verification requests through the admin dashboard.
- **Objective 3:** Develop a tutor searching and filtering system.
Status: Achieved. Students can search and browse verified tutors through the tutor search module.
- **Objective 4:** Implement a simplified booking system.
Status: Partially Achieved. The booking interface and backend APIs were implemented, but complete frontend-backend integration could not be finalized due to unresolved authentication and routing issues.
- **Objective 5:** Develop role-based dashboards.
Status: Achieved. Separate dashboards were successfully developed for Students, Tutors, and Administrators.
- **Objective 6:** Allow administrators to manage tutor verification requests and enable users to manage bookings.

Status: Partially Achieved. Tutor verification management was successfully implemented. Booking management functionality was partially completed because of the unresolved booking integration issue.

Overall, the project successfully demonstrates the core functionalities required for a tutor hiring platform and provides a solid foundation for future enhancements.

7.2 Challenges Faced

During the development of ShikkaLink, the team faced several challenges:

1. **Time Constraints:** The project was completed within a limited academic schedule. As a result, advanced features such as online payment, real-time communication, and availability scheduling were deferred to future work.
2. **Frontend-Backend Integration:** Integrating the frontend with backend APIs required significant debugging. Authentication, routing, and API communication issues occasionally caused inconsistencies during development.
3. **Booking Module Integration:** Although the booking interface and backend APIs were implemented, unresolved authentication token validation and routing issues prevented complete frontend-backend integration before project completion.
4. **Maintaining UI Consistency:** As multiple team members worked on different modules simultaneously, maintaining a consistent user interface required continuous collaboration and refinement.

7.3 Future Work

Several improvements can be incorporated into future versions of ShikkaLink:

- Resolve the remaining frontend-backend integration issues in the booking module.
- Implement tutor availability scheduling and time-slot management.
- Integrate secure online payment gateways.
- Add real-time chat and video conferencing between students and tutors.
- Develop AI-based tutor recommendation and matching.

- Introduce location-based tutor searching.
- Develop Android and iOS mobile applications.
- Add progress tracking, notifications, and learning analytics.

7.4 Individual Contribution

Each team member actively contributed throughout the project according to their assigned responsibilities.

Table 7.1: Individual Contribution

Member		Sprints	Modules Built	Signature
Maimuna (C233460)	Tabasum	1–8	Team leadership, sprint coordination, frontend-backend integration, module merging, system testing, and project management.	_____
Tahasina Afra (C233456)	Tasnim	1–8	Backend development, authentication APIs, tutor verification APIs, tutor search APIs, booking APIs, database integration, and backend testing.	_____
Safiyat (C233475)	Sayma	1–8	Frontend development, homepage, dashboards, tutor verification interface, tutor search, tutor profile, booking interface, UI/UX implementation, and interface refinement.	_____

References

1. Pressman, R. S. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
2. Sommerville, I. *Software Engineering*. Pearson Education.
3. Next.js Documentation. Available at: <https://nextjs.org/docs>
4. Node.js Documentation. Available at: <https://nodejs.org/docs>
5. Express.js Documentation. Available at: <https://expressjs.com>
6. Prisma ORM Documentation. Available at: <https://www.prisma.io/docs>
7. MySQL Documentation. Available at: <https://dev.mysql.com/doc>
8. JSON Web Token (JWT) Documentation. Available at: <https://jwt.io>
9. bcrypt Documentation. Available at: <https://github.com/kelektiv/node.bcrypt.js>
10. diagrams.net (Draw.io). Available at: <https://www.diagrams.net>

Appendices

Appendix A: Project Documents

This appendix contains the supporting project documents, including the project proposal, Software Requirements Specification (SRS), requirement elicitation questionnaire, interview notes, and other related documentation.

Google Drive Folder:

[https://drive.google.com/drive/folders/1YNSNu-n59v8ZVaum5z0ILfIE-JliUrb_]
]

Appendix B: GitHub Repository

The complete source code, commit history, project branches, and implementation files are available in the GitHub repository below: [<https://github.com/coder-maimuna/ShikkaLink--Tutor-Hiring-Platform.git>]

Appendix C: UI/UX Design (Figma)

The complete UI/UX design, including wireframes, high-fidelity designs, and the interactive prototype, is available through the Figma project.

Figma Design Link:

[<https://www.figma.com/design/CB0oTWV4fUsS0Ge105rG28/Shikkha-link-we-ui?node-id=0-1&t=0AkjiqdA4hhfFJQt-1>]

Appendix D: Frontend Deployment

The frontend of the ShikkaLink application was successfully deployed using Vercel. The deployment demonstrates the implemented user interface, including the home-page, authentication pages, dashboards, tutor search, tutor profiles, and other completed frontend components.

Due to time constraints and the project scope, the complete frontend-backend deployment could not be finalized before the completion of this report. The backend remains configured for local development and testing.

Vercel Deployment Link:

[<https://shikka-link-tutor-hiring-platform-pbecoetux-shikka-link.vercel.app/>]